

Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics

Muhammad Ibrahim Shoeb

MyHR: POLICY-AWARE MACHINE LEARNING FOR ANOMALY DETECTION IN WORKFORCE PROGRESSION

BME VIK Scientific Students' Association Conference (TDK) 2026

SUPERVISOR
Dr. Husam Al-Magsoosi

BUDAPEST, 2026

Contents

Student Declaration 5
Abstract 6
1 Introduction 7
1.1 Motivation 7
1.2 Problem Statement 7
1.3 Research Questions 7
1.4 Research Approach 8
1.5 Contributions and Non-Claims 9
2 Background and Related Work 10
2.1 Isolation-Based Anomaly Detection 10
2.2 Context and Policy Awareness 10
2.3 Explainability and Review Evidence 10
2.4 Evaluation Under Class Imbalance 11
2.5 Synthetic Data and Circularity 11
2.6 Historical Policy and Bitemporal Representation 11
2.7 Human Review and Duplicate Detection 12
2.8 Research Gap 12
3 MyHR and the Formal Progression Model 12
3.1 Application Boundary 12
3.2 Progression Arithmetic 13
3.3 Promotion, Sub-Race, and Annual Increment 13
3.4 Performance and Manual Approval 13
3.5 Administrative Evidence 13
3.6 Policy-Engine Parity 14
3.7 Temporal Representation 14
4 Research Methodology 14
4.1 Task Separation 14
4.2 Synthetic Record Contract 15
4.3 Data Partitions and Holdout Boundary 15
4.4 Original Scenario Matrix 16
4.5 Independent Administrative-World Simulator 16
4.6 Deterministic Checks 17
4.7 Feature Representations 17
4.8 Models and Statistical Baselines 18
4.9 Thresholds and Review Budgets 19
4.10 Metrics 19

4.11 Paired Scenario Analysis 19
4.12 Duplicate Detection 20
4.13 Formal Experimental Estimands 20
4.14 Reproducibility and Artifact Audit 20
4.15 Planned Blinded Realism-Review Method 21
4.16 Safety and Decision Boundary 21
5 Implementation and Reproducibility 21
5.1 Module Boundaries 21
5.2 Golden Histories and Tests 21
5.3 Literature and Claim Governance 22
5.4 Continuous Integration 22
5.5 Working Demonstration in MyHR 22
6 Development Experiments and Results 25
6.1 Interpretation Boundary 25
6.2 Protocol-V2 Validation Family 26
6.3 Original Rule-Negative Validation 26
6.4 Historical Independent Stress Test 27
6.5 Feature Collapse in the Historical Design 28
6.6 Historical-Policy Evidence Status 29
6.7 Corrected Exploratory Development Artifact 30
6.8 Result Summary 32
6.9 Robustness Slices and What They Can Show 33
6.10 Utility Sensitivity 33
7 Discussion and Threats to Validity 34
7.1 Answer to the Research Questions Today 34
7.2 Rules First, ML Second 34
7.3 Internal Validity 34
7.4 Statistical Validity 35
7.5 Construct and External Validity 35
7.6 Temporal Validity 35
7.7 Explanation and Human Factors 35
7.8 Privacy, Security, and Fairness 35
7.9 Reproducibility Versus Validity 36
7.10 Approved Redesign and Remaining Freeze Gate 36
7.11 Practical Implications for MyHR 36
8 Conclusion and Future Work 36
References 38

Appendix: Reproduction 39

Student Declaration

I, Muhammad Ibrahim Shoeb, declare that I prepared this paper myself without any unauthorized assistance, using only the sources indicated (literature, tools, etc.). Every part that I have taken from another source, either verbatim or rephrased with the same meaning, is clearly marked with a reference to its source.

I consent to BME VIK publishing the basic data of this work (author, title, abstract, year of completion, supervisor) in a publicly accessible electronic form, and the full text through the university's internal network or to authenticated users. I declare that the submitted work and its electronic version are identical.

Budapest, 1 November 2026

.....
Muhammad Ibrahim Shoeb

Abstract

Workforce progression systems apply promotion, commendation, sanction, and salary-increment policies to complex administrative records. Detecting erroneous records is challenging because unusual histories may be legitimate under the applicable policy. A short promotion interval, for example, may reflect a recorded commendation rather than an error. This research investigates whether incorporating explicit progression-policy context into machine-learning features can reduce false anomaly alerts while preserving detection of actual errors, thereby reducing manual HR review workload by prioritising records for evidence-based attention.

The study uses MyHR, an offline HR decision-support application, as the experimental platform. A formal progression model specifies service requirements, commendation credits, sanction delays, and annual increments. Deterministic validation addresses explicit contradictions, and Isolation Forest models rank the remaining histories using raw or policy-conditioned features at equal review capacity. Alerts support human review and never trigger personnel decisions.

Evaluation uses reproducible synthetic histories with injected errors and rare legitimate cases, separate training, validation, and test partitions, and wrong-policy, shuffled-policy, and ablation controls. In the audited 12-scenario development artifact, policy-aware minus raw recall at equal review capacity was +0.0278 (95% paired bootstrap interval from -0.0278 to +0.1019, sign-flip p=0.6875), and the wrong-policy control selected the same records as the policy-aware representation. In a later 900-scenario diagnostic, wrong-policy features reproduced most of the raw-to-policy gain. No policy-aware advantage was demonstrated, and the interval does not establish equivalence. All results are development evidence: the protocol is a draft, the final holdout has not been generated or accessed, no human realism review has been performed, and the optional user study was not conducted. Duplicate detection is implemented and evaluated separately, and its benchmark is incomplete.

Keywords: machine learning, anomaly detection, Isolation Forest, workforce progression, rule-based decision support, synthetic data, duplicate detection, explainability

1 Introduction

1.1 Motivation

Employee progression decisions are built from administrative facts that accumulate over time. Service duration, role level, performance evidence, commendations, sanctions, policy versions, and recorded approvals all affect eligibility for promotion or an annual salary increment. Executing explicit eligibility rules is straightforward; reviewing data quality is harder. Some errors are direct contradictions, such as an end date before a start date. Other histories are suspicious only in context: a promotion after a short interval may be an error, or it may be lawful because a commendation shortened the required service.

A detector that flags every unusual history creates unnecessary review work and may escalate legitimate exceptional cases, while one that suppresses too many alerts misses real errors. The goal is therefore a review queue, not automated personnel decisions. Explicit violations are explained deterministically, and less direct patterns are ranked for an authorized person. The scientific question is whether representing the applicable policy improves that ranking beyond raw event counts and intervals.

MyHR, a standalone Python and PySide6 desktop application backed by SQLite and SQLAlchemy, already contains an auditable progression model. It manages employee and organization data, performance reviews, promotions, commendations, sanctions, annual increments, imports, reports, and an application audit trail. Progression is calculated from source records rather than stored as an edited percentage, which gives research features and explanations a defined policy boundary.

1.2 Problem Statement

The research problem has three parts. Explicit contradictions should be detected reliably and explained with their evidence. Histories that are not explicit violations may still deserve review because their event patterns are unusual. Legitimate policy-dependent histories should not be treated as errors only because they are rare. No single accuracy score captures all three, so the problem needs separate deterministic and statistical components, comparisons at fixed review capacity, complete denominators, and a protected evaluation design.

Unsupervised anomaly detection is attractive when labelled errors are scarce, but it has no notion of what an organization considers lawful. Isolation Forest scores observations by how quickly random recursive partitions isolate them [1,2]. Its score measures statistical isolation, not administrative invalidity, and the same history can be anomalous under one policy and legitimate under another. This motivates policy-conditioned features, together with controls that test whether the supplied policy information is actually used.

1.3 Research Questions

The study is organized around three questions:

1. Does explicit policy context improve progression-anomaly ranking beyond raw administrative features and deterministic validation?

2. Does any observed advantage remain across employee populations, anomaly mechanisms, policy configurations, prevalence levels, and random seeds?
3. At equal review capacity, can policy context reduce false alerts on rare but legitimate histories without missing genuine administrative errors or merely moving false alerts to ordinary clean records?

Duplicate-record detection is a pairwise entity-resolution problem, not an employee-level progression anomaly, and is evaluated separately. Mixing it with progression labels would obscure both tasks and distort denominators.

1.4 Research Approach

The architecture applies rules first and machine learning (ML) second (Figure 1). Deterministic checks identify explicit date, event, and policy contradictions, and statistical baselines and Isolation Forest rank the remaining records. Raw and policy-aware representations share scenario, split, and model seeds, are compared at equal review budgets, and are accompanied by wrong-policy, shuffled-policy, and ablation controls that are reported even when unfavorable. Every alert is a recommendation to a human reviewer.

Synthetic histories provide known ground truth without private data, but a generator can encode the same rule the detector uses, so that the method succeeds by construction. The study therefore adds an independently implemented administrative-world simulator to the original policy-shaped generator and records causal truth separately from observed records. Independence of the generator is necessary but not sufficient; realism still requires human review.

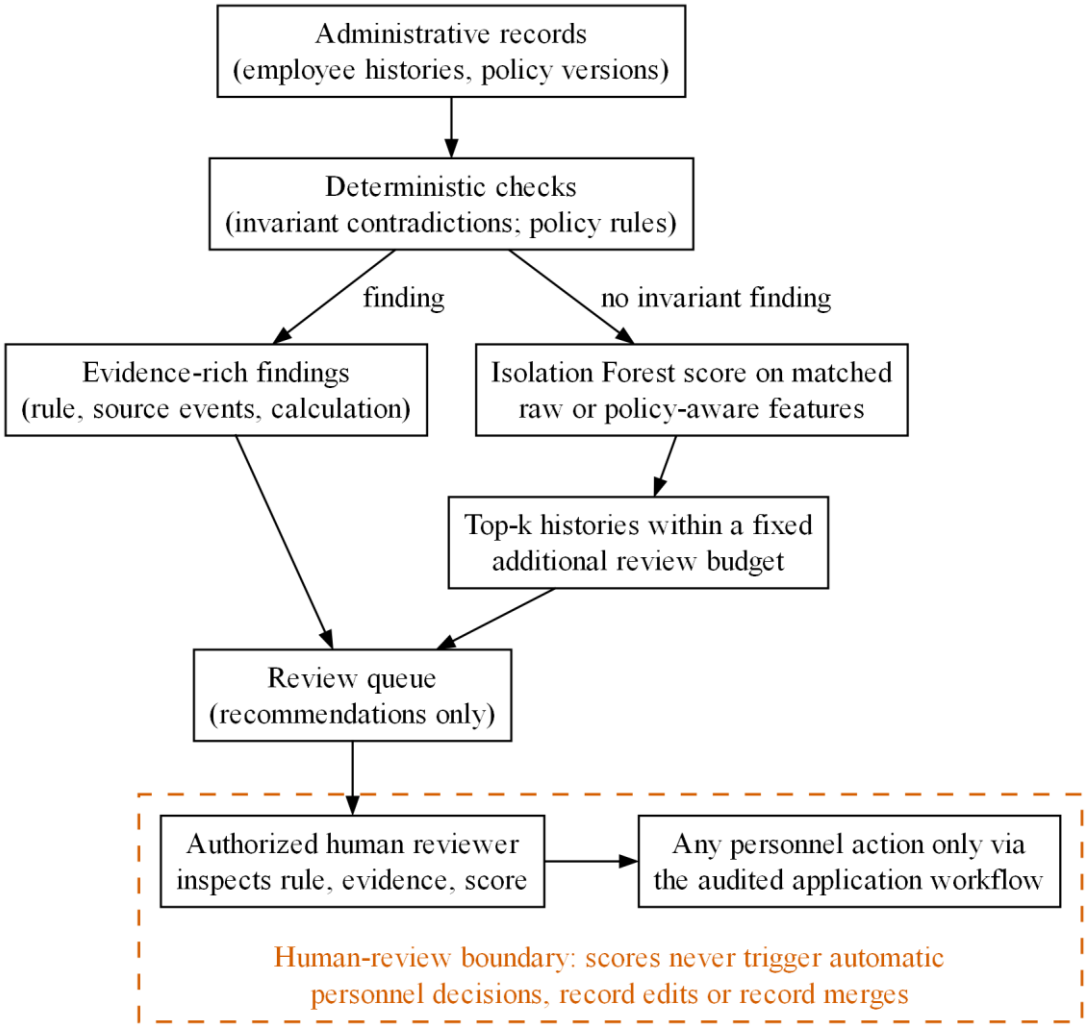


Figure 1. Rules-first, ML-second decision-support architecture. Histories without an invariant finding receive matched raw or policy-aware features, an Isolation Forest score, and a position within a fixed additional review budget. The dashed boundary marks the only decision point, an authorized human reviewer; scores never change personnel records.

1.5 Contributions and Non-Claims

The implemented work contributes a formal progression model shared with the application; evidence-rich deterministic validation; raw and policy-aware feature families with statistical and Isolation Forest baselines; validation-only thresholds and equal-budget comparisons; wrong-policy, shuffled-policy, and ablation controls; an independent development simulator; write-once, checksummed artifacts with independent recomputation of metrics; and a separate duplicate-candidate experiment.

The development evidence also yields a negative methodological result. In an early residual design, deterministic filtering consumed the policy signal and held all three policy-specific features at zero (Section 6.5). A later audit showed that apparent false alerts on policy-transition cases were responses to genuine contradictions created by the generator (Section 6.6). Both failures were preserved and led to the corrected design and stronger provenance checks.

The paper does not claim that policy-aware ML is generally better, that synthetic rates transfer to real organizations, or that the system should automate promotion, sanction,

salary, or record-merging decisions. The final holdout has not been generated or accessed.

Section 2 reviews related work. Section 3 describes MyHR and its progression model, and Section 4 the research methodology. Section 5 covers implementation, reproducibility, and the in-app demonstration. Section 6 reports the development experiments, Section 7 discusses their validity, and Section 8 concludes.

2 Background and Related Work

2.1 Isolation-Based Anomaly Detection

Isolation Forest rests on the observation that anomalies are often easier to isolate than ordinary points. Isolation trees are built by repeatedly choosing a random feature and split value, and the expected path length across trees becomes the anomaly score [1]. The later isolation-based formulation analyses the method more broadly and explains its computational behavior [2]. The method suits MyHR because it produces a continuous ranking, handles several numeric features, and needs no labelled anomalies during fitting.

A high score means that a feature vector is isolated relative to the fitted population, not that the record is invalid, unlawful, or harmful. MyHR therefore keeps deterministic policy evidence, model score, threshold, and human decision separate.

ADBench shows that anomaly-detection performance varies substantially with dataset, supervision assumptions, and method family [3], which argues for comparing any model against transparent baselines. The aim here is to measure what the policy representation contributes under matched conditions, not to find a universally best detector.

2.2 Context and Policy Awareness

Context-dependent anomaly detection concerns observations that are unusual only in a particular situation, and knowledge-based context can change how an observation should be interpreted [4]. Workforce progression is a clear case: the same elapsed service can comply with one policy version and violate another. Commendation credits, sanction delays, employee track, recognized prior service, and the effective policy period all change the expected progression path.

Policy-aware features are therefore transformations relative to an explicit rule set, such as a service deficit relative to the applicable requirement. They can align the model with administrative meaning, but they can also leak the answer if the generator uses the same residual to define anomalies. A genuine policy signal should weaken when the policy identity is wrong or shuffled, which is what the controls test.

2.3 Explainability and Review Evidence

Explainable anomaly-detection research separates global understanding of a detector from local evidence for a specific alert [5], and DIFFI derives feature importance for Isolation Forest from tree depth and feature use [6]. Feature importance can describe a model score, but an HR reviewer also needs administrative evidence: policy version, required and observed service, adjustments, source records, and deterministic findings.

MyHR gives priority to this administrative evidence; a model attribution, if added later, would be labelled as model evidence rather than a policy fact. Explanation quality is a separate evaluation problem. The registered abstract makes a user study optional, and this paper claims no usability results.

2.4 Evaluation Under Class Imbalance

Administrative anomalies are expected to be rare. Under severe class imbalance, receiver-operating-characteristic summaries can look favorable while the precision of the review queue is poor, and precision-recall analysis is often more informative [7]. Average precision (AP) summarizes a ranking without a test-selected threshold, but it depends on prevalence, so every AP value needs the number of ranked records and positive cases.

Threshold metrics are still needed because HR teams review a finite number of cases; Sections 4.9 and 4.10 define the operating points and measures used here. Metric choice follows the review task because anomaly-detection metrics encode different assumptions, a point made for time-series detection in [8].

The evaluation protocol can itself inflate results. Train-test partitioning choices, thresholds tuned on the test set, and contamination rates taken from the test data can all inflate F1 and average precision [9], and conclusions may depend on a narrow evaluation pipeline [10]. Selecting favorable seeds is a related risk, which MyHR addresses by fixing and reporting all seeds. MyHR records separate dataset, split, and model seeds, selects thresholds on validation data, retains negative results, and keeps the final holdout locked until the protocol is frozen.

2.5 Synthetic Data and Circularity

Synthetic data allows controlled injection of known errors and rare legitimate histories, but realistic evaluation requires more than plausible-looking rows. Steinbuss and Böhm show the importance of varying synthetic outlier characteristics and data domains [11]. MyHR additionally treats defining anomalies through the detector's own decisive feature as a circularity risk; this is a design inference applied to the present experiment, not a prescription attributed to that paper. Data-centric work on synthetic tabular data likewise emphasizes downstream behavior and multiple profiles over a single similarity score [12].

The original MyHR generator and the policy-aware detector share policy concepts, so results on that generator are useful for engineering but vulnerable to circularity. The independent simulator (Section 4.5) avoids shared code, but source-code separation does not ensure conceptual separation, as the historical residual design showed (Section 6.5).

2.6 Historical Policy and Bitemporal Representation

Administrative systems must distinguish when a fact or policy is effective in the modeled world from when it is recorded in the database; bitemporal research calls these valid time and transaction time [13]. The distinction supports historical reconstruction, corrections, and auditability, and applies directly to promotions decided under an earlier policy, recognized service recorded later, and corrections entered after an event. A preserved MyHR artifact shows the failure caused by replaying current policy over constructed histories intended to represent lawful historical exceptions (Section 6.6), and the corrected research adapter replays the applicable historical policy (Section 3.7).

2.7 Human Review and Duplicate Detection

Human-in-the-loop anomaly detection treats expert validation as part of the workflow instead of assuming that model output is true [14]. MyHR keeps the same boundary, and its proposed realism review is blinded to model scores and generated labels so that reviewers judge plausibility rather than confirm the detector.

Duplicate detection is evaluated as entity resolution. Candidate generation and pair scoring must be separated, because scoring every possible pair makes false-positive rates look deceptively small, and dense-blocking research stresses candidate workload and blocking recall [15]. This secondary task provides no evidence for the progression-anomaly hypothesis.

2.8 Research Gap

The audited literature covers Isolation Forest, contextual anomaly detection, imbalanced evaluation, the limits of synthetic data, bitemporal history, explanation, and entity resolution. It does not provide a standard benchmark for policy-aware workforce progression. The contribution is therefore bounded: formalize one progression domain, build reproducible controls, test whether policy context changes ranking at matched workload, and document when the experiment cannot answer that question. The performed search does not support a claim that no prior system combines these elements.

3 MyHR and the Formal Progression Model

3.1 Application Boundary

MyHR is a local-first desktop system with a PySide6 interface and SQLite persistence through SQLAlchemy. Research dependencies are kept out of the packaged application. The operational modules (Section 1.1) form the context for the research package, which reads structured representations and cannot change personnel data.

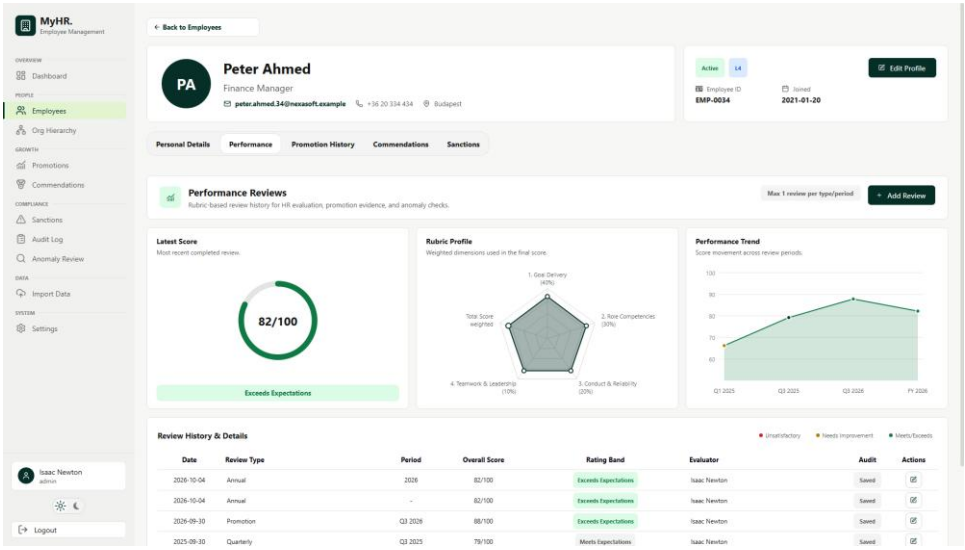


Figure 2. MyHR employee-management interface used as the application context for policy evidence and human review.

3.2 Progression Arithmetic

Let B denote the base number of service months required for the next level, M the completed months since the last promotion or recognized start point, C the months of valid commendation credit, and S the months of active sanction delay. Adjusted service A and remaining service R are defined as:

```
A = M + C - S
R = max(0, B - A)
```

Service-requirement status and bounded progress are:

```
service_requirement_met = (R = 0)
progress = clamp(floor(100 A / B), 0, 100)
```

The form makes the sign of each adjustment explicit: a commendation brings the employee closer to the requirement, and a sanction delays eligibility. The implementation records the source events and policy configuration used in each calculation. Promotions remain manual decisions even when the service requirement is met.

3.3 Promotion, Sub-Race, and Annual Increment

A promotion race represents service toward the next level. Sub-race checkpoints track annual progress and salary-increment timing but are not promotions. Employees in the `Other` track have no degree-based promotion race, although they receive annual increments, commendations, sanctions, and performance reviews, so the system does not invent a promotion path for roles governed by different rules.

Annual salary increments are independent of promotion: completing a yearly interval can make an increment due when the employee is not eligible for a new level, whereas a promotion changes level and salary through an explicit approval workflow.

3.4 Performance and Manual Approval

Performance reviews use a weighted rubric covering goal delivery, role competencies, conduct and reliability, and teamwork and leadership. The score is evidence shown during promotion review and does not replace policy eligibility, since a candidate may complete a race with no recent review or with poor performance. The confirmation dialog shows policy status, latest performance, active sanctions, commendations, salary change, warnings, and a link to the employee profile; the authorized reviewer approves or cancels, and every outcome is audited.

3.5 Administrative Evidence

The research representation separates employee facts, administrative events, policy epochs, generated truth, and observed records. This matters when an observation is incomplete or late: a missing increment import may be a data error although the underlying event occurred, and a sanction linked to the wrong employee changes the observation ledger but not the generated world. The independent generator records these distinctions for new artifacts; the original immutable artifact retains only its historical schema.

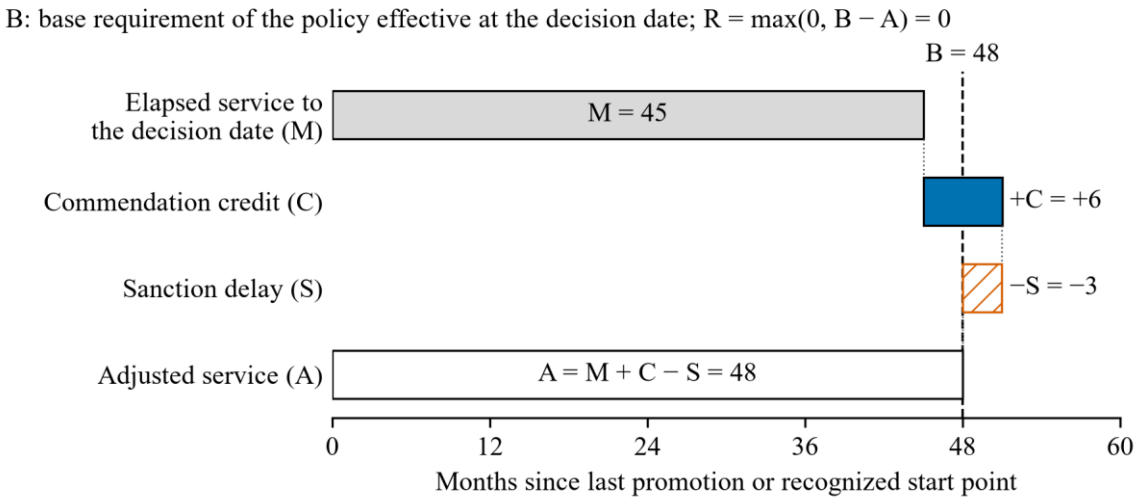


Figure 3. Example progression timeline showing base service, commendation credit, sanction delay, and the need to retain the policy effective at the decision date.

3.6 Policy-Engine Parity

Research calculations must agree with the application on shared histories. The application calls a pure promotion-race calculation shared with research code, and golden-history tests (Section 5.2) cover clean progression, commendation credit, sanction delay, promotion reset, completed service, and non-progression tracks. Passing these tests shows implementation agreement on the tested histories, not that the policy itself is organizationally correct.

3.7 Temporal Representation

Replaying the current policy is insufficient for historical decisions. The research record therefore retains policy identity and version, valid and recorded times, jurisdiction, recognized service, approval and entitlement evidence, and correction or retry provenance. Development code links grandfathering evidence to the employee, policy version, and event, and provides two temporal views. The `known_then` view restricts effective facts, recorded facts, and policy knowledge to the decision date. The `later_review` view keeps the same effective decision horizon but admits evidence and amendments recorded by the review date. These semantics are tested research code; they are neither a migrated production schema nor part of a frozen protocol.

4 Research Methodology

4.1 Task Separation

The primary task ranks employee histories for progression-related review. Positive classes are inconsistent dates, unexplained frequent promotions, excessive sanctions, and unusual administrative intervals. A broadly labelled rare-legitimate class contains histories that are unusual but intended to be lawful under the generated policy context, and clean records include both ordinary and exceptional non-error histories.

Duplicate detection is a separate pairwise task whose positive unit is a pair or entity cluster and whose identity attributes do not belong in progression features. Reporting it separately keeps the many easy non-duplicate pairs from diluting progression metrics and keeps duplicate labels from changing anomaly prevalence.

4.2 Synthetic Record Contract

Each synthetic history contains an employee identifier, track, service and role dates, promotions, commendations, sanctions, annual increments, performance evidence, and the administrative timestamps the scenario needs. Generated labels and causal truth are excluded from prediction features. Dataset, split, and model seeds are recorded separately, and all records of one latent person stay in one partition for duplicate detection.

The taxonomy (table below) distinguishes the nominal scenario pattern from the mechanism realized in each record. Generation applies explicit compatibility and record-level eligibility checks instead of silently replacing an infeasible pattern. Stable-administration and merger/migration populations may host all five rare patterns; rapid expansion excludes legacy policy transitions because of its hire window. Future artifacts preserve nominal and realized values, complete event fields, entitlement evidence, policy epochs, and whether the causal event was observed or omitted; existing artifacts are not rewritten.

Category	Example mechanism	Expected evidence	Evaluation role
Inconsistent dates	Event before employment start	Direct chronological contradiction	Deterministic baseline and anomaly label
Frequent promotion	Promotion before adjusted service requirement	Promotion interval and applicable policy	Policy check and anomaly ranking
Excessive sanctions	Dense or repeated sanction history	Event count, timing, resolution, service delay	Statistical and policy context
Unusual interval	Abnormally short administrative gap	Event intervals and process context	Statistical ranking
Wrong employee link	Sanctions attached to another employee	Observation ledger differs from generated world	Independent residual mechanism
Missing import	Increment event omitted from observation	World event exists but record is absent	Independent residual mechanism
Approval bypass	Promotion without required approval evidence	Explicit workflow violation	Deterministic check
Batch retry	Duplicate or repeated administrative events	Repeated event references and intervals	Deterministic check
Rare legitimate	Previous policy, recognized service, or delayed recording	Temporal policy and evidence context	False-alert analysis

4.3 Data Partitions and Holdout Boundary

Seeds 2026 to 2030 were inspected during early feature development and are permanently development-only. Protocol-development scenarios may be generated and inspected to diagnose implementation and design failures, and validation scenarios may inform threshold selection and feature-family decisions. The final holdout will not be defined, generated, inspected, or scored until the protocol, scenario matrix, feature families, thresholds, and analysis are frozen with human approval.

The boundary is enforced in code: the protocol runner refuses a holdout run purpose, and existing metadata records `holdout_accessed: false`. A future one-time holdout run

must use a frozen source revision and a complete manifest. Any later change would require a new protocol version and a new holdout family, with the original result preserved.

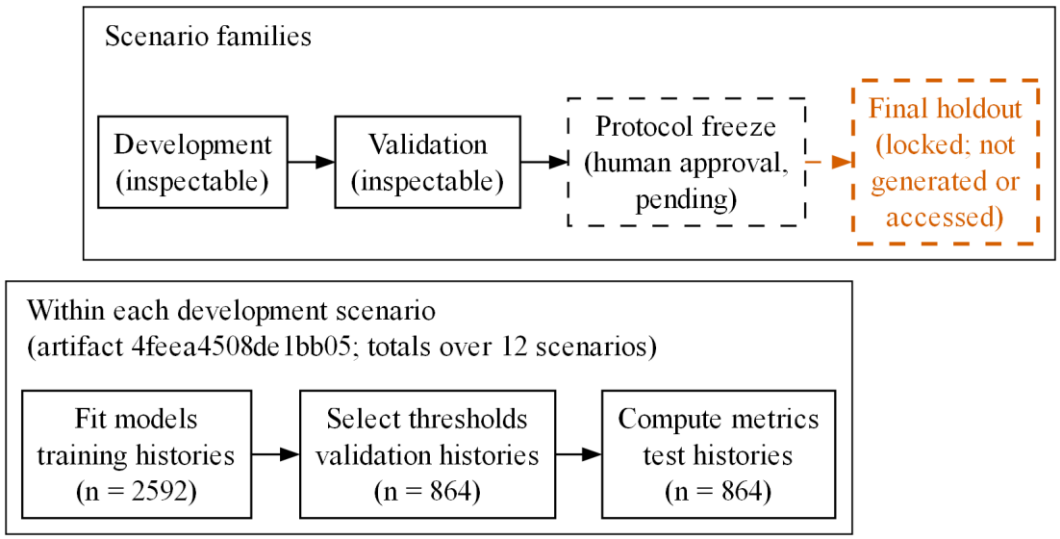


Figure 4. Evaluation lifecycle. Within each scenario, models are fitted on training histories, thresholds are selected on validation histories, and metrics are computed on test histories (counts from artifact 4feea4508de1bb05). The final holdout has not been generated or accessed and stays locked until a human-approved protocol freeze.

4.4 Original Scenario Matrix

The protocol-v2 generator varies policy profile, population profile, anomaly mechanism, rare-legitimate pattern, prevalence, and seeds, and produces matched raw and policy-aware representations of the same records. Thirty development and thirty validation scenarios were generated with 600 employees and 120 test records per scenario, giving 3,600 validation test histories.

This generator is useful for testing pipeline mechanics, but it is conceptually close to the detector: several injected anomaly mechanisms become direct policy deficits, so the policy-aware representation can mirror the generator's construction. Its results are development diagnostics, not transferable evidence.

4.5 Independent Administrative-World Simulator

The independent simulator uses only the Python standard library and does not import detector features, deterministic checks, the application policy engine, or the original generator. It first constructs an administrative world and then derives an observation ledger that can contain omissions, wrong links, or retries. Twelve development scenarios combine three population families with four causal error mechanisms and rare patterns. The author approved lawful-case eligibility rules for protocol preparation: a late correction requires a progression-track employee with an existing promotion and valid supersession; a policy transition additionally requires completion of the 30-month legacy service period by 31 December 2022; and grandfathering requires entitlement evidence linked to the employee, policy version, and relevant event. The balanced allocation of protocol-scale cells is not yet frozen. The source-code boundary removes one form of leakage, but filtering and feature construction can still erase the policy information under test, so the independent run is a stress test, not a holdout.

4.6 Deterministic Checks

Deterministic validation returns findings with their evidence rather than a single boolean. Policy-agnostic checks cover date contradictions and duplicate event references, and flag unusually dense administrative activity for review, since a busy period is not by itself invalid; policy checks compare progression service with the supplied policy. Rules are reported both as a standalone baseline and as the first stage of a rules-plus-ML queue.

Explicit findings are never dropped to meet a nominal capacity. The original cascade uses a 10% capacity floor in which all deterministic findings are mandatory and ML fills any unused capacity, which complicates equal-budget comparison when rules alone exceed the floor. The approved primary redesign applies identical invariant checks to both pipelines and gives each an equal, fixed additional ML budget over the same invariant-negative histories. The complete policy-rules queue is reported as a secondary result.

4.7 Feature Representations

Raw features describe recorded activity without evaluating it against a policy: event counts, sanction and commendation counts, promotion and increment intervals, and administrative density. The policy-aware representation adds to a matched common core quantities such as service deficit, deficit ratio, and an unexplained-short-promotion indicator.

For a completed promotion with required service B, elapsed service M, commendation credit C, and sanction delay S, a conceptual policy deficit D is:

```
D = max(0, B - M - C + S)
deficit_ratio = D / B
```

The sign convention matches the progression equation: credits reduce remaining service and sanctions increase it. The implementation evaluates this expression over the recorded history and event evidence rather than in isolation.

Controls (Figure 5) include raw features without the commendation-count shortcut, single-feature representations that add only one policy feature (or only the raw promotion interval) to the common core, wrong-policy context, and shuffled-policy context. They are needed because extra columns can improve a model through added information without correct use of the policy.

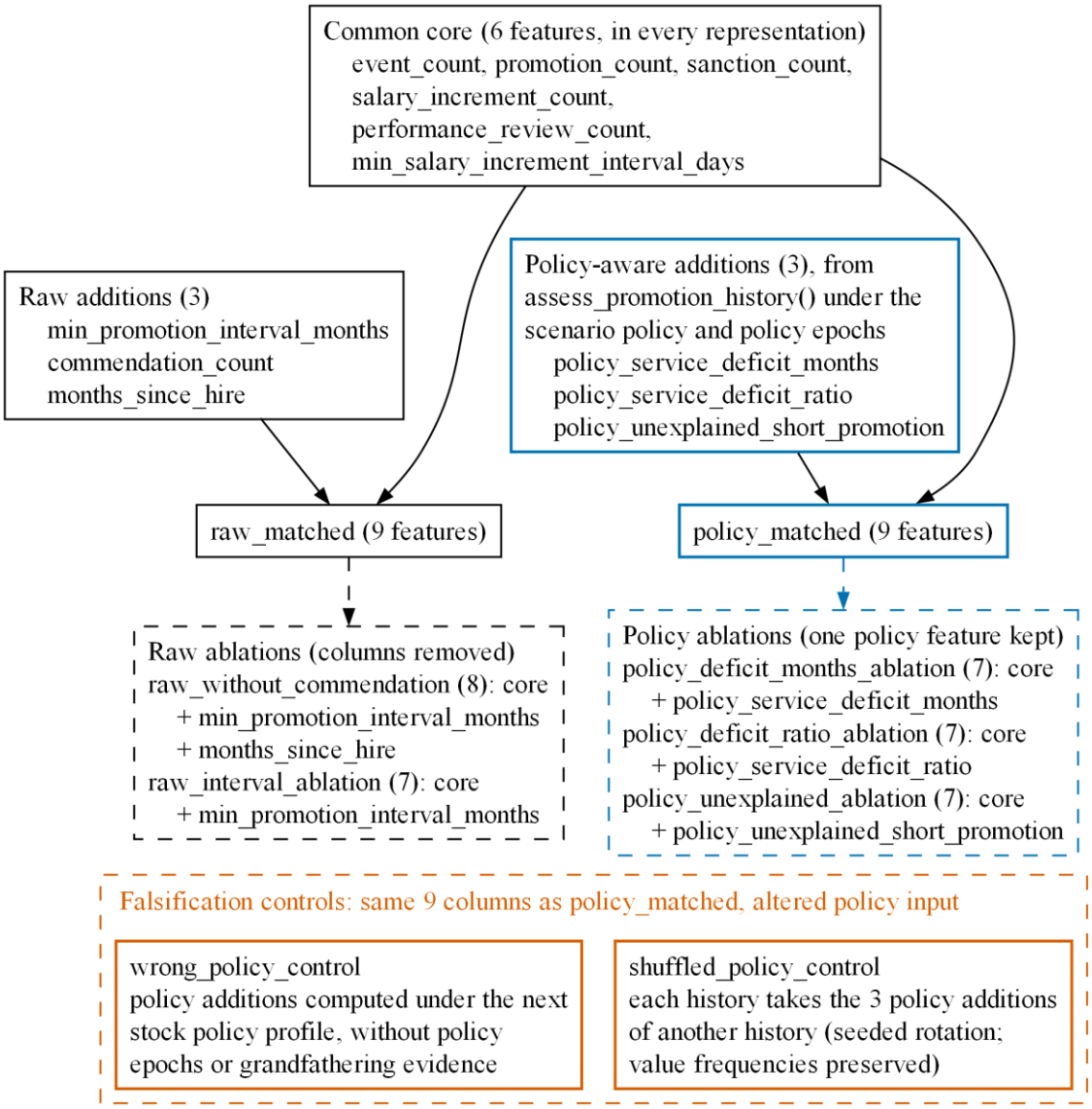


Figure 5. Matched feature representations and controls as defined in the research code. Raw and policy-aware representations share a six-feature common core and add three features each. Single-feature representations keep the core plus one feature; the wrong-policy and shuffled-policy controls use the policy-aware columns with altered policy input.

4.8 Models and Statistical Baselines

Interquartile-range and z-score methods are the transparent statistical baselines. Isolation Forest is fitted with continuous scores and `contamination="auto"`, so configured test prevalence is never passed in as a threshold. Within a scenario, every representation receives the same training records and seed budget.

Larger scores mean higher review priority. Predictions are produced at predefined operating points, AP is recomputed from the stored continuous scores, and runtime is recorded for every method.

4.9 Thresholds and Review Budgets

Four views are retained. Average precision evaluates ranking without a test-selected threshold. A fixed nominal top-10% operating point compares methods at equal review capacity; the queue size is `ceil(0.10 n)`, so with 72 test records per scenario it is 8 records (a realized 11.11%), or 96 alerts across 12 scenarios. A target false-positive-rate threshold is selected only on validation data. A validation-maximized F1 threshold is secondary and gets the same search budget for every representation.

Alongside the percentage, the paper reports alert counts, true and false positives, missed anomalies, and rare-legitimate and ordinary-clean alerts, which shows whether an improvement for one group only moves false alerts to another.

4.10 Metrics

For true positives TP, false positives FP, false negatives FN, and true negatives TN:

```
precision = TP / (TP + FP)
recall = TP / (TP + FN)
F1 = 2 * precision * recall / (precision + recall)
false_positive_rate = FP / (FP + TN)
```

Recall and AP are undefined for a scenario with no positive cases. Such scenarios remain in design and denominator tables and are excluded only from paired inference for those metrics, under a predeclared eligibility indicator. Older immutable artifacts that stored zero keep that historical convention, which cannot define the confirmatory endpoint.

Operational measures are anomalies found per 100 reviews, rare-legitimate reviews per 1,000 employees, missed anomalies per 1,000 employees, runtime, and total workload. Review cost, missed-error cost, and rare-escalation cost are sensitivity assumptions, not measured organizational values (Section 6.10).

4.11 Paired Scenario Analysis

Raw and policy-aware methods are paired within each scenario seed. For metric m and scenario i:

```
delta_i = m(policy-aware_i) - m(raw_i)
```

Scenario seeds, not employees, are the independent units, because employees generated in one scenario share policy and population assumptions. Every included protocol-scale condition uses 30 independent seeds unless a written power or feasibility justification approves another count before the run. Development tooling reports mean and median paired differences, scenario bootstrap intervals, and paired sign-flip tests. Fixed-capacity recall is the single primary endpoint. The confirmatory secondary family contains AP, rare-legitimate false-alert rate, and ordinary-legitimate false-alert rate for the policy-aware versus raw comparison, with Holm's step-down adjustment [16,17] across these three two-sided tests. Precision and total false alerts are descriptive companions at fixed capacity. Controls, ablations, threshold analyses, per-class results, and utility sensitivity are exploratory.

4.12 Duplicate Detection

The duplicate experiment generates latent people and derives noisy records from them. It compares exact matching, normalized exact matching, and weighted similarity, with the similarity score:

```
similarity = 0.35 name + 0.30 date_of_birth
           + 0.25 identifier + 0.10 email
```

The score proposes candidate pairs for review and never merges records. A defensible final benchmark must separate blocking recall from pair scoring, use entity-disjoint partitions, include hard non-duplicates, and select its threshold on validation data. The existing run is preliminary secondary evidence only.

4.13 Formal Experimental Estimands

Let s identify a generated scenario (its profiles, mechanisms, prevalence, and seeds), E_s the histories remaining after identical invariant checks, and Y_s their binary anomaly labels. A representation r and method m produce continuous scores a_{s,r,m} over E_s.

At fixed additional review capacity k_s, let A_s,r be the top-k_s alert set. The primary scenario-level recall difference is:

```
delta_recall_s = recall(Y_s, A_s,policy) - recall(Y_s, A_s,raw)
```

Equal capacity requires |A_s,raw| = |A_s,policy| over the same E_s, and the complete rules queue is kept separate from this budget, so a representation cannot look better by raising more alerts. AP(Y_s, a_s,r,m) is a secondary ranking metric. Scenarios with no positive Y_s follow the eligibility rule of Section 4.10 and are never assigned zero or removed after score inspection.

With L_s the set of legitimate exceptional histories, the rare-legitimate flagged rate is |A_s,r intersect L_s| / |L_s|. It does not replace total false positives, because a method can reduce alerts in L_s while increasing them among ordinary-clean histories, as Section 6.3 shows.

The development code implements this estimand. The historical artifact, whose policy-specific features are constant, remains invalid for it, and a clean-revision development and validation run, an independent audit, and a protocol freeze are still required.

4.14 Reproducibility and Artifact Audit

Every experiment directory records the protocol status, full configuration, command line, Git revision, dirty-state flag, source hashes, Python and dependency versions, seeds, policy configuration, record-level predictions, scenario and aggregate metrics, and a SHA-256 manifest. Artifact directories are write-once and are published atomically from a staging directory only after successful completion (Figure 9).

The independent auditor checks manifests, source hashes, confusion counts, rates, AP recomputed from stored scores, aggregate consistency, paired differences, bootstrap intervals, sign-flip results, and correction fields. Passing the audit shows that the arithmetic is reproducible, not that the estimand, scenario population, or policy assumptions are valid (Section 7.9).

4.15 Planned Blinded Realism-Review Method

Code and invariant tests check synthetic correctness; organizational realism needs human review. The prepared packet contains fictional histories without model scores, generated labels, or method names. For each history, reviewers would judge plausibility, status (ordinary, legitimately exceptional, erroneous, or indeterminate), evidence sufficiency, confidence, and whether escalation is appropriate.

The method uses two independent reviewers with complementary HR-policy and information-systems expertise so that disagreement is visible. The predeclared sample has 48 cases: 12 ordinary legitimate, 12 rare legitimate, 12 administrative error, and 12 ambiguous or evidence-insufficient histories, balanced across available populations, policy versions, and compatible mechanisms. Both reviewers receive the same cases in independently randomized order, blinded to labels, model outputs, and each other's first pass. Raw agreement and unweighted Cohen's kappa with a case-bootstrap interval are reported for record status. Initial ratings are preserved before joint evidence-based adjudication, and unresolved cases remain ambiguous. Cases may not be rewritten to improve model metrics. The reviewer identities and qualifications, sampling output, seeds, and packet checksum are still pending, and the packet has not been distributed.

4.16 Safety and Decision Boundary

The anomaly pipeline ends at a review recommendation. A high score does not establish misconduct, poor performance, ineligibility, or duplicate identity, and deterministic findings can be wrong if the supplied policy or history is incomplete. The interface must therefore show the applicable rule, source events, calculation, model signal, uncertainty, and reviewer action separately. Promotion approval, sanction and salary changes, user management, imports, and record edits are transactional and audited with before-and-after evidence, and duplicate candidates are never merged automatically. The evaluation measures prioritization quality, not autonomous personnel outcomes.

5 Implementation and Reproducibility

5.1 Module Boundaries

Research services live under `src/services/anomaly_detection`, and the independent scenario code under `src/research`. Synthetic data structures, feature extraction, deterministic checks, statistical baselines, Isolation Forest scoring, duplicate detection, metrics, and protocol evaluation are separate modules. The desktop application does not inherit scikit-learn, pandas, or plotting dependencies.

The separation reduces packaging risk, lets policy logic be tested without research models, and allows adapters to be audited for leakage and the independent generator to be checked for prohibited imports.

5.2 Golden Histories and Tests

Golden-history tests run the same progression cases through the pure policy arithmetic and the SQL-backed application, which also verifies adapter behavior. Further tests cover deterministic checks, generator determinism, feature values, threshold selection, wrong-policy and shuffled-policy controls, write-once artifacts, manifest tampering, source-hash mismatches, citation governance, and holdout refusal.

Adversarial tests were added whenever an audit found a gap, for example to reject incomplete causal event joins, truth-only prediction columns, unmanifested citation artifacts, evidence bound to the wrong paper, and overwrite attempts. A successful happy-path run cannot show that the audit boundary works.

5.3 Literature and Claim Governance

Literature discovery uses free direct clients for OpenAlex, arXiv, Crossref, and Semantic Scholar, with optional API keys kept outside Git. Each reference is deduplicated with its identifiers and retrieval provenance recorded, and each scientific claim is linked to an evidence locator. An independent citation audit on 29 October 2026 validated 18 references and 15 claim records; metadata corrections recorded on 1 November 2026 await independent re-audit. Sixteen legally accessible full texts are preserved locally.

Two references remain metadata-only: Holm's multiple-comparison paper [17] and Krippendorff's reliability paper [18]. The Holm procedure claim is supported through Demšar's full-text description [16]; the reliability claim attributed to Krippendorff remains marked unverified, and metadata verification is not treated as scientific evidence.

5.4 Continuous Integration

GitHub Actions runs the Python test suite on pushes and pull requests, GitGuardian checks for exposed secrets, and citation schema and governance checks run without private credentials. Large research artifacts that do not suit every hosted CI run are audited locally.

CI shows only that the checked code and tests pass; it cannot approve a hypothesis, interpret a negative result, or authorize a holdout run.

5.5 Working Demonstration in MyHR

The desktop application includes a synthetic-only Anomaly Review workflow, so the pipeline can be inspected without reading source code or JSON artifacts. A reviewer selects a labelled development scenario and starts the analysis; the interface reports model-fitting, reserved validation, eligible evaluation, and review-queue counts separately. The run calls the same adapter, invariant checks, policy-history assessment, feature representations, Isolation Forest scorer, and fixed-capacity evaluation functions as the research services; the interface contains no second detector.

The review queue (Figure 6) separates deterministic contradictions from statistical suspicion and hides generated truth. Selecting an item opens the employee's evidence (Figure 7): source events, effective and recorded dates, policy ID and version, elapsed and required service, commendation credit, sanction delay, adjusted service, deficit, and evidence limitations. Generated labels appear only in a separate Synthetic Evaluation view, which reports injected-error, lawful-rare, and ordinary counts with a per-case list hidden by default.

The live comparison (Figure 8) gives the raw, policy-aware, wrong-policy, and shuffled-policy representations the same eligible residual records and additional review capacity, and charts recall, precision, and queue composition from the same result object used for the tables and the CSV and PDF exports. A model exception is shown as a failed run instead of silently returning a rules-only queue. Leaving the page during a

run terminates the worker and discards the partial result; there is no separate cancel button. The workflow performs no database write or personnel action.

The demonstration is not a research experiment. It runs three fixed development scenarios with 120 synthetic employees each (72 fitting, 24 validation, and 24 evaluation histories), a 25-tree Isolation Forest, and a review capacity of 3 of 24 records (nominal 10%, realized 12.5% after rounding). The table below lists the outcomes produced by `scripts/verify_anomaly_demo.py` and the demonstration service at the reviewed revision.

Scenario	Population	Raw TP/FP	Policy-aware TP/FP	Wrong-policy TP/FP	Shuffled-policy TP/FP	Rare legitimate alerted (each method)
`ind-dev-01`	Stable administration	2/1	3/0	2/1	2/1	0/2
`ind-dev-07`	Rapid expansion	0/3	0/3	0/3	0/3	0/2
`ind-dev-12`	Merger and migration	0/3	0/3	0/3	0/3	0/2

In-app demonstration outcomes at equal review capacity (24 evaluation records, 3 reviews per method). Each scenario has 3 injected errors among its evaluation records.

The first scenario is shown by default and in the figures. Its policy-aware queue finds all three injected errors, but this is a single realization of 24 records. In the other two scenarios no representation, including the controls, finds any injected error within three reviews. The demonstration shows that the pipeline runs end to end and that its outputs are traceable. These are inspected development observations that help to debug and demonstrate the pipeline. They are too few to support any conclusion about the research hypothesis.

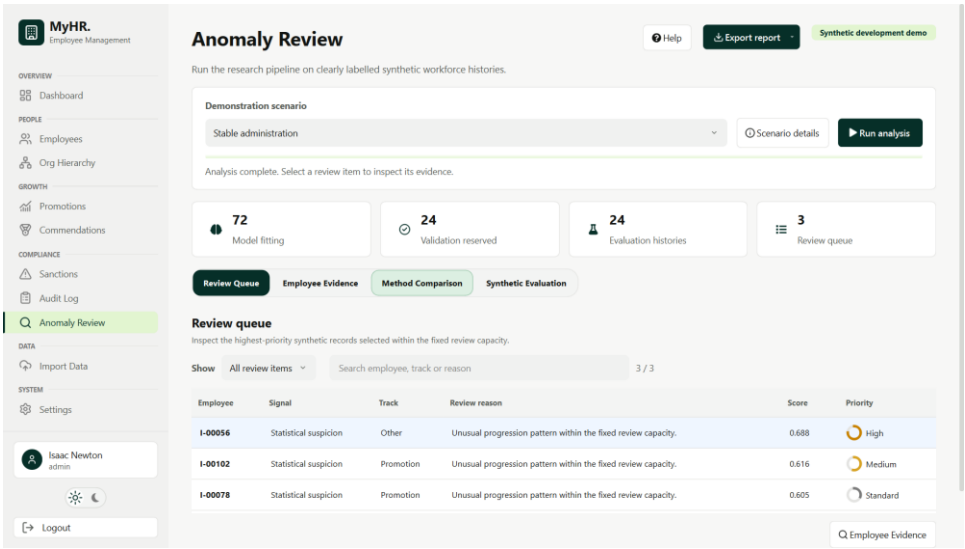


Figure 6. Synthetic Anomaly Review queue after running the shared rule and Isolation Forest services. Generated truth is not shown in this operational view.

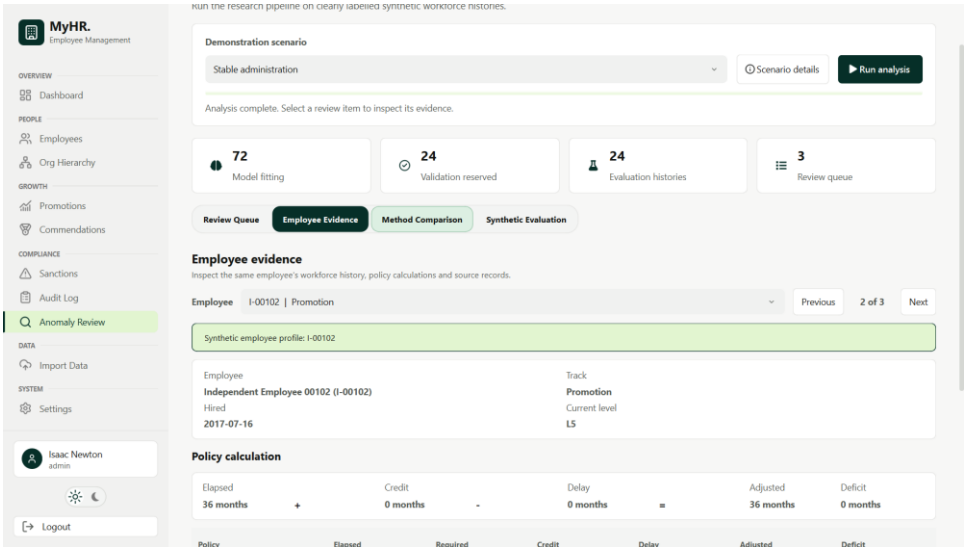


Figure 7. Employee Evidence view for queue item I-00102, showing the profile and the service-adjustment arithmetic; the source events with effective and recorded dates are in the expandable history below.

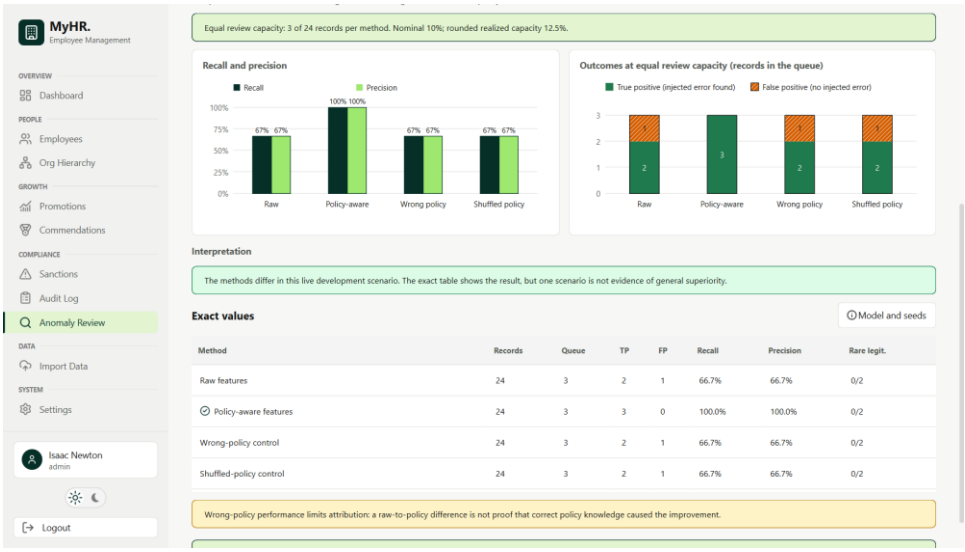


Figure 8. Method Comparison view for demonstration scenario ind-dev-01 (24 evaluation records, 3 reviews per method). Both charts and the exact table come from the same completed run. The wrong-policy and shuffled-policy controls are always shown; one 24-record scenario is not evidence of a policy effect (Section 6.7).

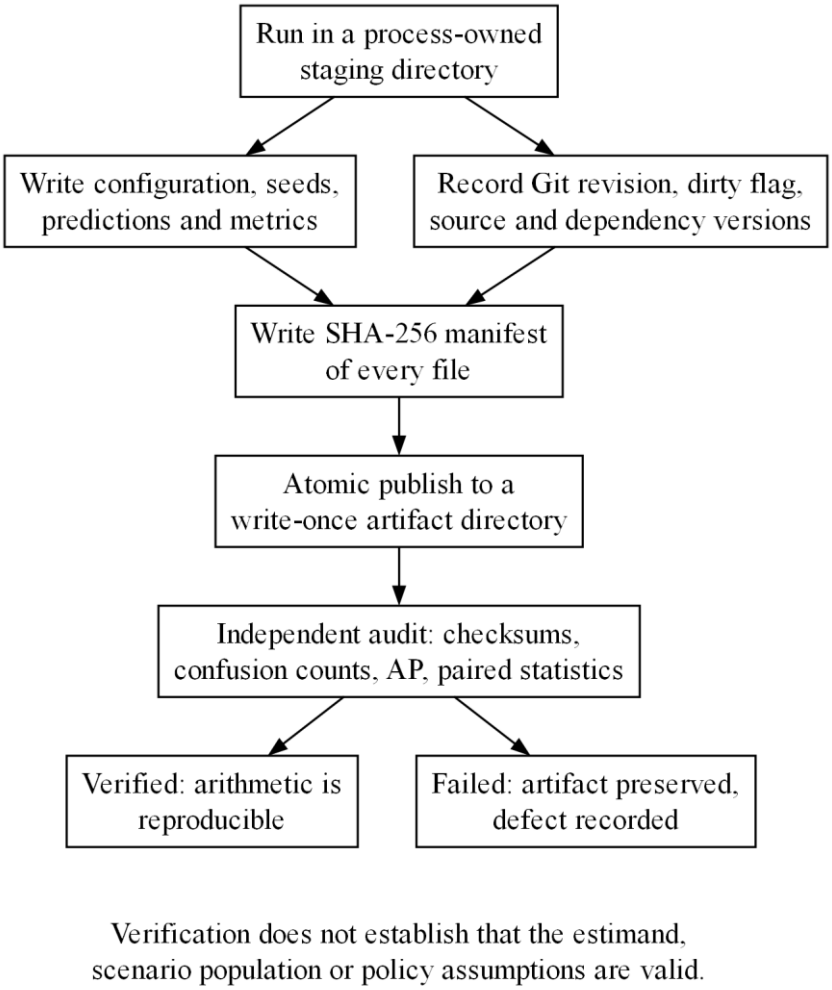


Figure 9. Write-once artifact lifecycle. Runs record provenance and a SHA-256 manifest before atomic publication; an independent audit recomputes checksums and statistics, and failed artifacts are preserved with the defect recorded. Verification establishes reproducible arithmetic, not scientific validity.

6 Development Experiments and Results

6.1 Interpretation Boundary

All results in this section are development or validation diagnostics that were inspected during feature and protocol work and can never support confirmatory claims. They expose generator coupling, feature collapse, and temporal-policy failure; they do not establish deployment accuracy or a policy-aware advantage. Sections 6.2 to 6.6 cover the original generator and the historical independent run, Section 6.7 the current corrected artifact, and Section 6.8 a later 900-scenario diagnostic.

Not reported here are numerical results for the IQR and z-score baselines, the duplicate-detection comparison against exact and normalised exact matching, and runtime measurements. These methods are implemented in the pipeline, but this paper neither analyses nor verifies their results, so it makes no claim about them.

6.2 Protocol-V2 Validation Family

The corrected validation artifact contains 30 scenarios and 3,600 test histories: 360 labelled anomalies, 3,240 non-anomalies, 200 rare-legitimate histories, and 3,040 ordinary-clean histories. Deterministic policy rules found 351 of 360 anomalies with no false positive. This recall partly reflects generator interaction, since several injected excessive-sanction histories also become explicit service-policy violations.

Validation method	Alerts	TP	FP	FN	Scenario-mean F1	Scenario-mean AP
Deterministic policy rules	351	351	0	9	0.986	0.977
Rules plus raw Isolation Forest	408	357	51	3	0.907	0.998
Rules plus policy-aware Isolation Forest	408	358	50	2	0.910	1.000

At the same total alert count, the policy-aware cascade finds one more anomaly and raises one fewer false alert. The paired mean AP difference is +0.0016 with a stored 95% bootstrap interval from 0.0000 to 0.0041, sign-flip p = 0.25, and exploratory adjusted p = 1.0, which is not a meaningful advantage. The near-perfect AP also reflects how closely the original generator is tied to the policy features.

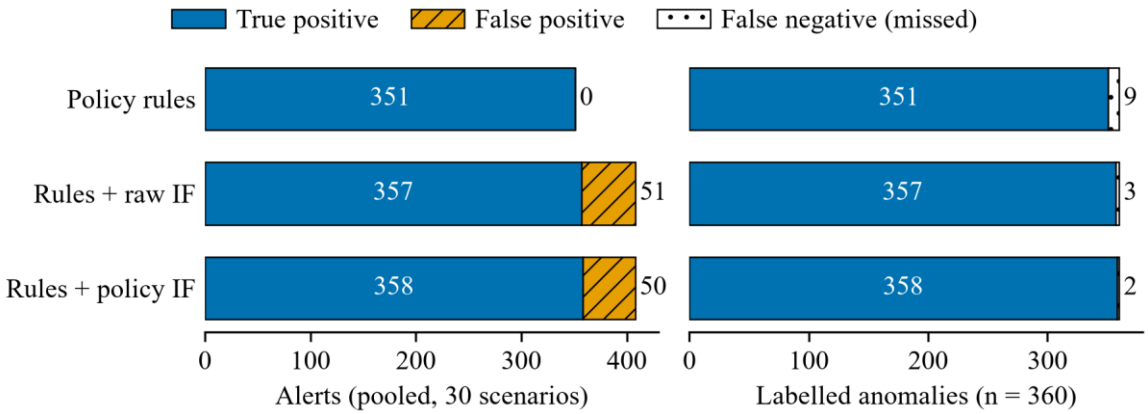


Figure 10. Protocol-v2 validation alert outcomes (artifact validation_c5878ca7b927ccd2), pooled across 30 scenarios and 3,600 test histories containing 360 labelled anomalies. IF denotes Isolation Forest in the rules-first cascade with a 10% capacity floor. The high rules performance is partly a generator interaction.

6.3 Original Rule-Negative Validation

Removing every deterministic policy finding leaves 9 labelled anomalies among 3,249 histories. At the nominal 10% budget, both residual models raise 340 alerts, find all 9 anomalies, and produce 331 false positives.

Residual validation method	Alerts	TP	FP	FN	Mean F1	Mean AP	Rare-legitimate alerts
Raw Isolation Forest	340	9	331	0	0.047	0.161	59/200
Policy-aware Isolation Forest	340	9	331	0	0.047	0.181	27/200

Policy-aware features avoid 32 alerts among rare-legitimate histories, but the same number of false alerts moves to ordinary-clean histories. Precision, recall, and review workload do not improve. Subgroup false-alert counts therefore cannot be interpreted without total false positives and capacity.

The paired AP difference is +0.0194 with a stored interval from -0.0417 to 0.0889. Twenty-four of the thirty rule-negative scenarios contain no residual positive case and store AP as zero under the historical convention, so the scenario means and interval depend on that convention (Section 4.10). The original generator leaves too few informative residual scenarios to estimate the incremental value of ML, and no alternative eligibility rule was chosen after inspection.

6.4 Historical Independent Stress Test

The first independent simulator run produced 12 test scenarios with 864 histories, 108 labelled anomalies, and 72 broadly labelled rare-legitimate histories, and recorded 39 deterministic-policy alerts among the rare histories (interpreted in Section 6.6). Its duplicate truth was later found invalid. The residual-model numbers below are kept to explain why the design was rejected; they are not current performance evidence.

After deterministic filtering, 759 histories remain, 42 labelled anomalies and 717 negatives. At equal nominal budget, both models raise 81 alerts.

Independent residual method	Alerts	TP	FP	FN	Mean F1	Scenario-mean AP
Raw Isolation Forest	81	26	55	16	0.304	0.3616
Policy-aware Isolation Forest	81	25	56	17	0.298	0.3621

The stored paired AP difference is +0.0005 with interval [-0.1173, 0.1079], sign-flip $p = 1.0$, and adjusted exploratory $p = 1.0$. These values are reproducible under the implemented convention but are evidence of neither equivalence nor a policy effect, for the reason given in Section 6.5.

6.5 Feature Collapse in the Historical Design

In the historical residual design, the policy-aware representation contained a six-feature common core plus policy service deficit, policy deficit ratio, and an unexplained-short-promotion indicator. The residual filter kept only records with no policy finding, and the same policy assessment emitted a finding whenever the service deficit was positive. Every retained record therefore had zero for all three policy-specific features.

The policy-aware model thus had no varying policy signal: its varying inputs were the six-feature common core, whereas the matched raw model kept three additional varying raw features. Ranking differences cannot be attributed to policy context, and the design could not test the primary hypothesis. This is a structural failure, not an inconclusive result, because the explanatory variable was absent after filtering rather than estimated with uncertainty. The run was preserved, the design diagnosed, and a new estimand submitted for approval instead of tuning features against inspected outcomes.

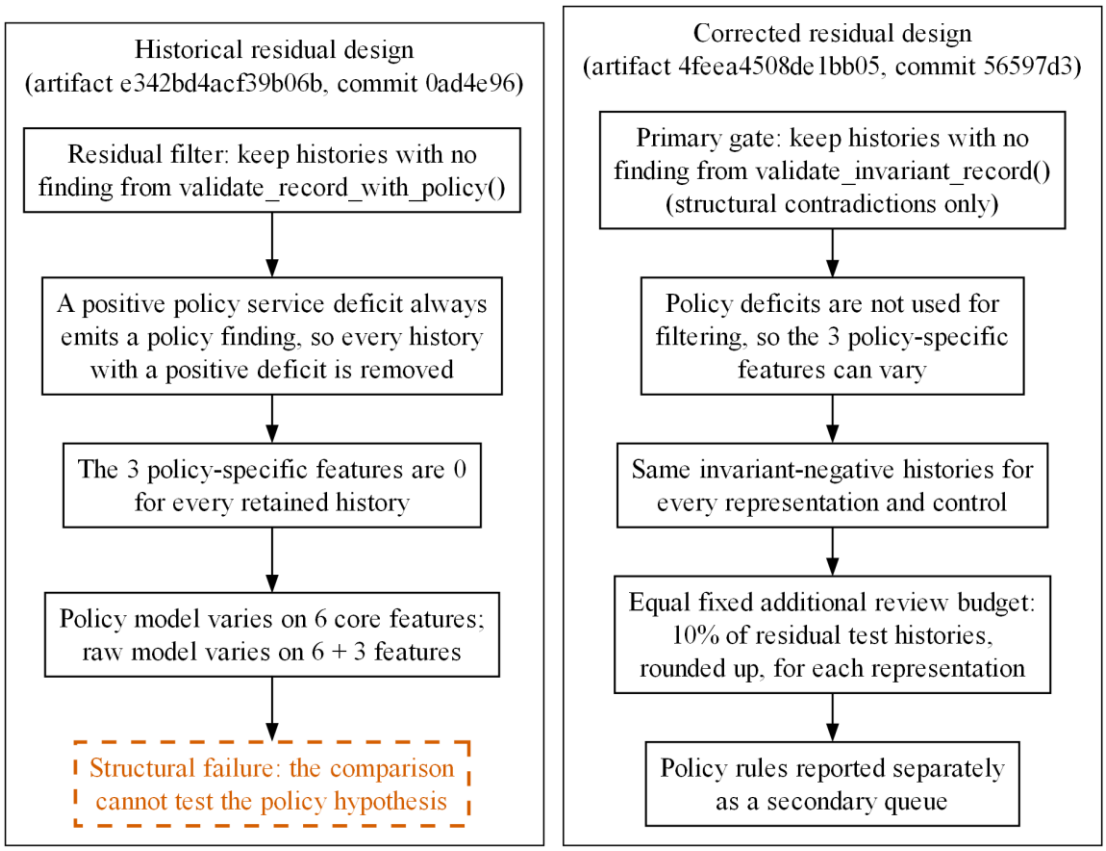


Figure 11. Historical design failure and corrected architecture. The original rule gate consumed the signal intended for ML; the repaired design limits the primary gate to invariant contradictions and assigns equal additional ML review capacity.

Six of the twelve scenarios also contain no residual positive case. The implementation stores AP as zero for them, whereas the approved design treats recall and AP as undefined (Section 4.10). Including these structural zeros halves both reported AP means and changes the uncertainty. Restricting the calculation to the six informative scenarios still gives a very wide interval and no advantage. The approved rule must be implemented and frozen before any confirmatory scenario is generated.

6.6 Historical-Policy Evidence Status

The preserved `39/72` and later `8/72` readings have different evidentiary status. Artifact `e342bd4acf39b06b` directly supports the historical count `39/72` for `deterministic_policy_rules` across broadly labelled rare-legitimate test histories; its separate `deterministic_rules` result is `0/72`. At source revision `0ad4e96`, the generator constructed 30-month grandfathered and 24-month recognized-service promotions with narrative evidence, which the adapter then evaluated with current-policy arithmetic. The result is a designed specification counterexample; because the artifact does not preserve employee-specific entitlement evidence or the realized mechanism per record, it is neither a prevalence estimate nor a mechanism-specific rate. The later `8/72` reading has a different problem. The second audit (`TDK_SECOND_EXTERNAL_AUDIT_2026-10-30.md`) showed that the policy-transition generator behind it created genuine contradictions, including promotions before hire and insufficient service. The count itself is reproducible. Only that `8/72` false-positive interpretation is invalidated. The generator was then repaired so that transition cases are lawful by construction.

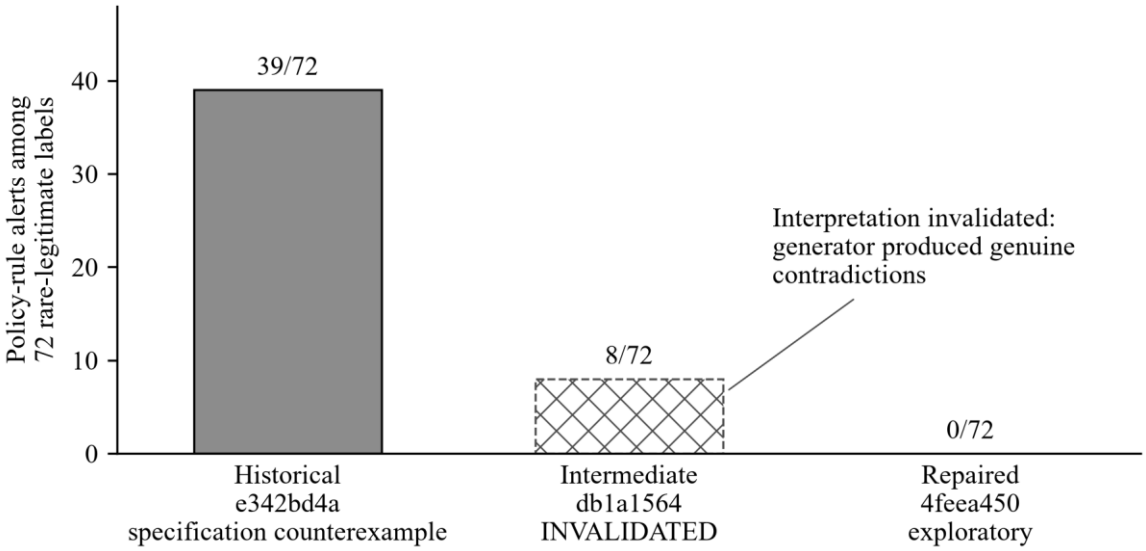


Figure 12. Deterministic policy-rule alerts among 72 generated rare-legitimate labels in three development artifacts. The historical 39/72 is a constructed specification counterexample, not a prevalence estimate. The intermediate 8/72 is hatched because its false-positive interpretation was invalidated by a generator defect. The repaired 0/72 denominator includes 24 grandfathering labels without independently established employee entitlement.

The audit still motivates temporal modeling as a requirement, but the experiments do not quantify its real-world frequency or benefit. Current development records represent the temporal and entitlement fields listed in Section 3.7, and separate decision-time and later-review replay is implemented and tested. Protocol-scale generation and independent verification of those paths are incomplete.

6.7 Corrected Exploratory Development Artifact

Artifact `4feea4508de1bb05` was generated from clean commit `56597d3` after the semantic and UI-lifecycle repairs. The strengthened artifact audit reports `verified` (stored arithmetic and provenance under the legacy schema; labels, entitlement, and temporal semantics are not independently validated), 12 scenarios, 864 eligible test histories, 108 anomalies, 72 generated rare-legitimate labels, 900 metric rows, 64,800 prediction rows, and `holdout_accessed: false`. Deterministic policy checks produced 63 true positives, no false positives, and 45 false negatives, and flagged none of the 72 generated rare-legitimate labels. The denominator includes 24 grandfathering labels for which employee-specific entitlement was not independently established, so `0/72` is not a validated false-positive rate on proven lawful cases.

Equal-budget residual representation	Alerts	TP	FP	FN	Generated rare labels alerted
Raw matched	96	36	60	72	9/72
Policy matched	96	39	57	69	6/72
Wrong-policy control	96	39	57	69	6/72
Shuffled-policy control	96	33	63	75	7/72
Core + policy deficit months only	96	42	54	66	8/72
Core + policy deficit ratio only	96	42	54	66	8/72
Core + unexplained-short-promotion flag only	96	42	54	66	8/72
Core + raw promotion interval only	96	42	54	66	9/72
Raw without commendation shortcut	96	40	56	68	9/72

The policy-matched model and the wrong-policy control selected identical alert sets across all 864 evaluated records. Each single-feature representation, which adds one policy feature to the common core, found 42 anomalies, against 39 for the full three-feature policy representation; the core plus the raw promotion interval alone also found 42. These adverse controls rule out attributing the small raw-to-policy difference to correct policy context, and they show that the policy feature set needs redesign and independent validation.

At the primary equal review capacity, policy-aware minus raw residual recall was +0.0278 across 12 paired scenarios, with a paired 95% bootstrap interval of [-0.0278, +0.1019] and sign-flip p-value 0.6875. The precision difference was +0.0313 with interval [-0.0313, +0.1042]. Average precision favored the policy representation in this inspected run, but AP is secondary and the broad exploratory multiplicity-adjusted p-values are 1.0. The artifact also predates the approved entitlement and temporal-view

contract and is below the required repeated-seed scale. No superiority, inferiority, or equivalence claim follows.

The corrected duplicate evaluation declares an explicit evaluation universe and records set membership for every TP, FP, FN, and TN. Exact methods use the complete partition-local pair universe, and similarity methods report blocking recall separately from metrics within their declared candidate universe.

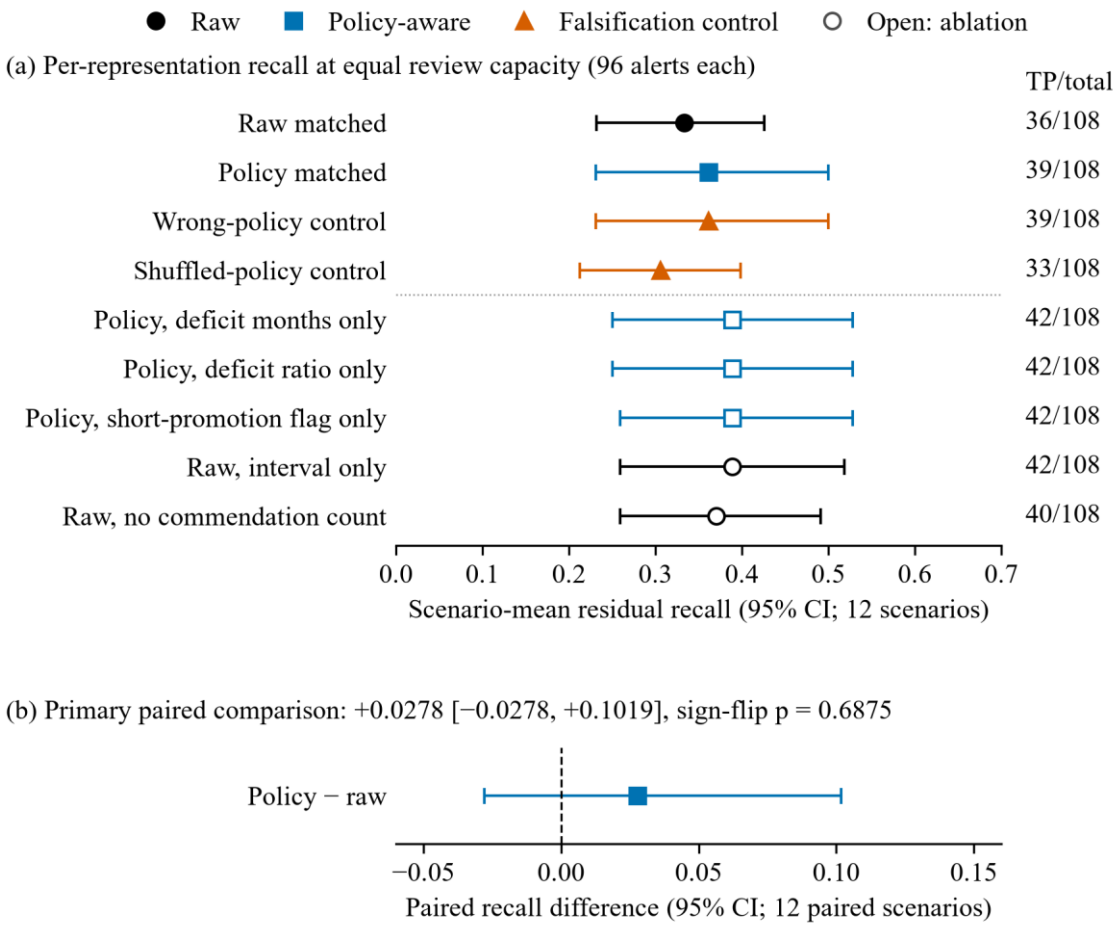


Figure 13. Residual recall at equal review capacity in exploratory artifact 4feea4508de1bb05 (12 scenarios, 864 eligible test histories, 108 labelled anomalies, 96 alerts per representation). (a) Scenario-mean recall with stored 95% intervals for every representation, including controls and ablations. (b) Paired policy-matched minus raw-matched difference with its 95% bootstrap interval and sign-flip p-value. The interval crosses zero.

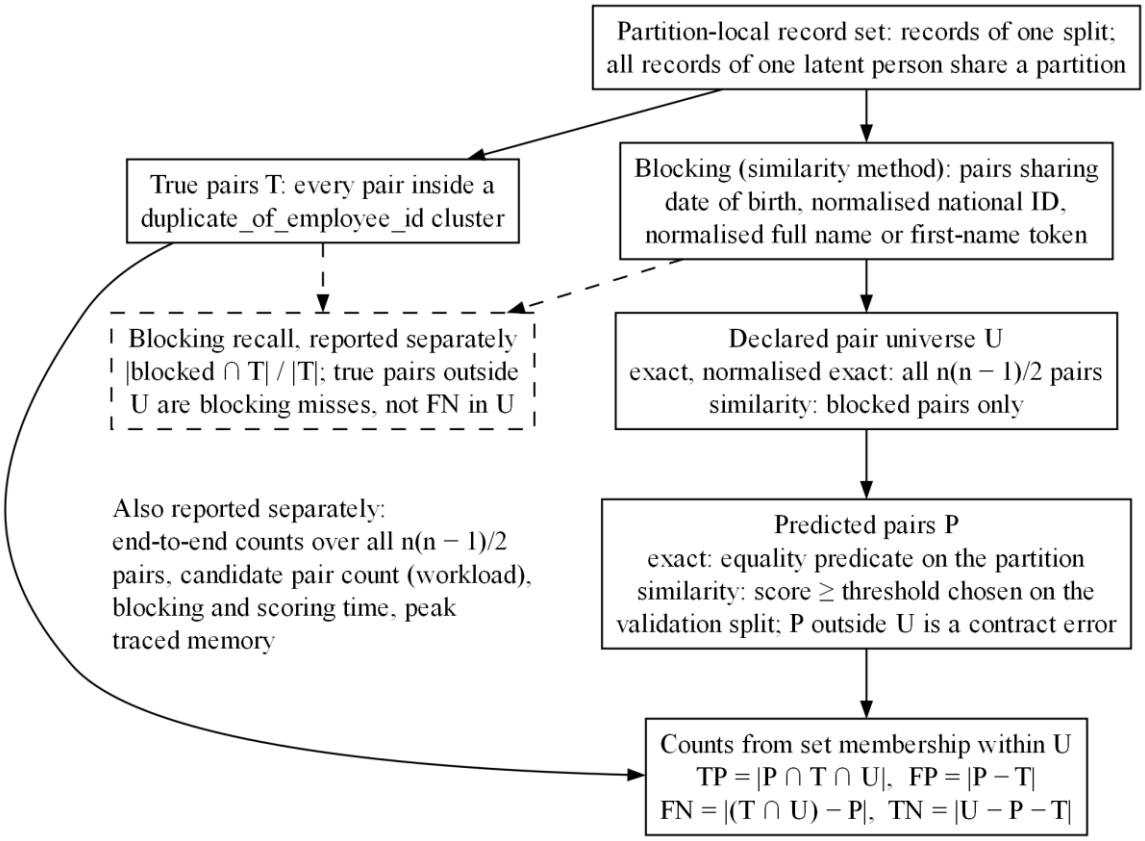


Figure 14. Duplicate-evaluation contract after repair. Pair ground truth is entity-contained; blocking recall, candidate-universe classification, end-to-end matching, workload, runtime, and memory are reported separately. No final duplicate-performance claim is made.

6.8 Result Summary

The historical runs show that passing tests and reproducible arithmetic are not enough when labels or evaluation universes are wrong. The corrected 12-scenario artifact runs reproducibly, but its primary interval crosses zero. A later development-only run used 30 seeds per condition and produced 900 scenarios in artifact `8f1494e4abf2d268` with `holdout_accessed: false`. Its policy-aware minus raw recall difference was +0.348, but wrong-policy features reproduced +0.284, about 82% of the apparent gain (the ratio of the wrong-policy minus raw to the policy minus raw mean recall difference, not a causal share). The remaining increment is still confounded with the information content and variance of the added columns (both representations have nine columns, but equal counts do not equalise scale), and the full offline audit of its 8,019,000 prediction rows is incomplete, so this diagnostic is not confirmatory evidence. The current evidence supports the framework, its controls, and its auditability, but not a policy-aware performance claim.

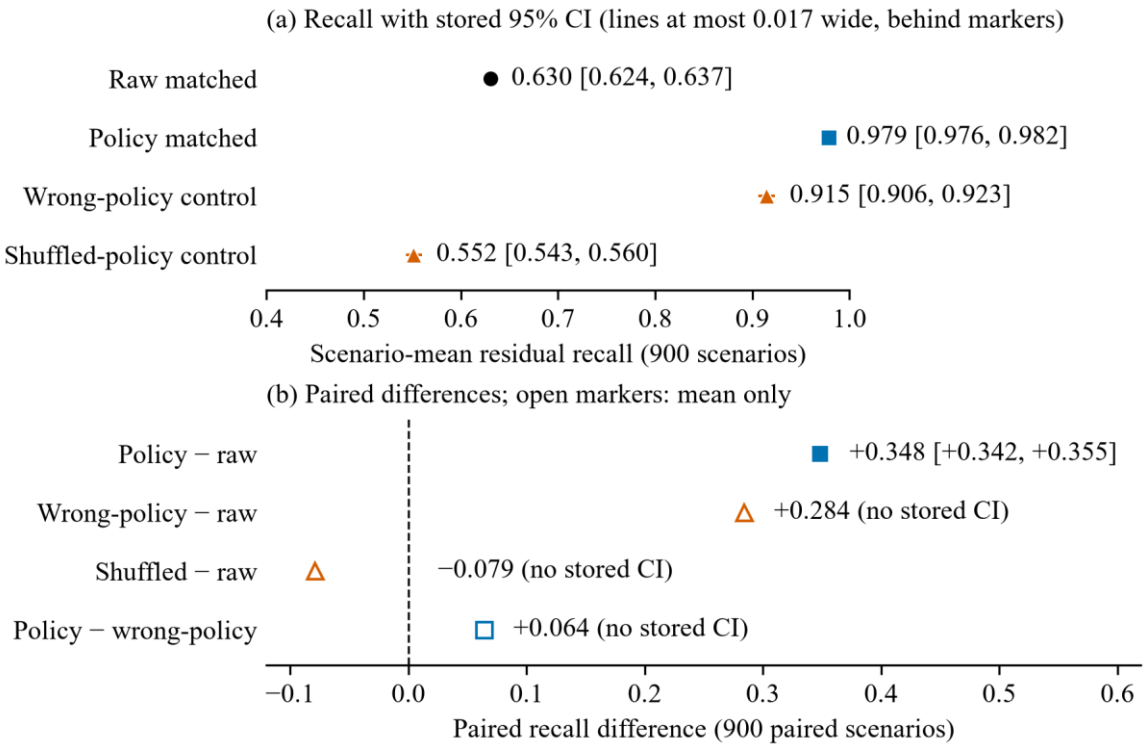


Figure 15. Confounded 900-scenario development diagnostic (artifact 8f1494e4abf2d268, 30 conditions with 30 seeds each, holdout not accessed). (a) Scenario-mean residual recall at the fixed review budget with stored 95% intervals. (b) Paired differences against raw features; only the policy-minus-raw difference has a stored interval, and the control differences are paired means recomputed from per-scenario recall. Wrong-policy features reproduce about 82% of the apparent gain.

6.9 Robustness Slices and What They Can Show

The stored artifacts include slices by policy profile, population profile, anomaly mechanism, rare pattern, prevalence, and operating point. They help diagnose whether a method's behavior is concentrated in one constructed condition, but slices that reuse the same scenario histories are not independent experiments. The paper neither counts each slice as a new sample nor selects the most favorable one.

Mechanism-specific residual comparisons describe the surviving rule-negative task, not the full taxonomy. The repaired artifact preserves per-mechanism counts and feature-variation checks, but with one random realization per population-mechanism cell, twelve scenarios cannot separate population, mechanism, rare pattern, and chance. These slices are diagnostics, not subgroup effects; estimating such effects needs repeated seeds within each cell (Section 7.4).

6.10 Utility Sensitivity

The pipeline stores a transparent cost model instead of a single claimed business value:

```
total cost = review_count * review_cost
           + false_negatives * missed_error_cost
           + rare_legitimate_alerts * rare_escalation_cost
```

Each cost is a sensitivity parameter.No utility sensitivity results are reported here; the proposed evaluation would show how conclusions change across cost ratios instead of choosing one ratio that favors a method. The current comparison does not justify a utility advantage: the primary recall interval crosses zero, and any favorable utility

result would depend on cost weights that must be declared and reviewed rather than hidden inside a score.

7 Discussion and Threats to Validity

7.1 Answer to the Research Questions Today

The first research question (RQ1, Section 1.3) is not answered positively. The audited 12-scenario interval crosses zero, and in the 900-scenario diagnostic a deliberately wrong policy reproduces most of the apparent gain (Section 6.8). This points to coupling between representation and policy structure rather than to a policy effect, and no superiority, inferiority, or equivalence conclusion follows.

RQ2, robustness across policies, populations, and seeds, cannot yet be assessed: the 900-scenario diagnostic used repeated seeds, but its confound is unresolved and its audit incomplete. A balanced compatible-cell matrix, a raw control with added non-policy columns of comparable variance, independent validation, and human realism review are needed first.

RQ3, on false alerts for rare legitimate histories, is also unanswered. Reducing alerts in one legitimate subgroup can simply move them elsewhere (Section 6.3), and generated lawful status must be checked semantically before subgroup metrics are interpreted (Section 6.6), so a valid experiment must report both subgroup and total false alerts at equal workload.

7.2 Rules First, ML Second

Direct contradictions should not be rediscovered by an opaque model. Deterministic checks give stable evidence, ML ranks residual patterns that are unusual without being violations, and the reviewer can tell a factual policy conflict from a statistical suggestion.

The boundary needs care: if rules remove every history in which policy residuals vary, a residual model cannot test those residuals (Section 6.5). On 29 October 2026 the project author therefore authorized a redesign that restricts the primary residual filter to invariant contradictions, compares raw and policy-aware ML at equal additional review capacity, and reports the complete rules-plus-ML queue as a secondary result. On 30 October the author approved the lawful scenario compatibility, entitlement, temporal-view, repeated-seed, and multiplicity rules documented in the decision addendum. Their implementation status and the remaining freeze steps are given in Section 7.10.

7.3 Internal Validity

The main internal-validity threat is circularity between generator and detector (Section 2.5). Wrong-policy, shuffled-policy, and ablation controls help but do not certify independence; the separate simulator reduces code reuse, and the blinded review is meant to assess realism. A future holdout should be specified first as business histories and then translated into records by an implementation isolated from detector code.

Threshold leakage is controlled by validation-only selection, and test prevalence is not passed to Isolation Forest. Seed and scenario choices are recorded, and negative and adverse results are retained. Scenario eligibility, compatibility, and zero-positive handling are predeclared, but the exact balanced cell list and seeds must still be frozen.

7.4 Statistical Validity

The 12-scenario independent artifacts have few units: in the historical design only six scenarios contained residual positives. Population, mechanism, rare pattern, and random realization cannot be separated cleanly, and the bootstrap intervals assume an exchangeable scenario population that is not yet defined. The current intervals are descriptive development output.

The draft protocol therefore requires 30 independent seeds per included protocol-scale condition unless a written power or feasibility justification is approved before execution (Section 4.11). Repeated seeds within compatible cells separate scenario factors from random realization, and the single primary endpoint with one predeclared three-test Holm family limits selective interpretation.

7.5 Construct and External Validity

Synthetic labels establish correctness relative to the generator, not organizational truth. No private employee data were used, which protects privacy but limits external validity. Feature ranges, policy transitions, missing-data mechanisms, and utility costs need domain calibration. The 48-case blinded review protocol, agreement measure, and adjudication rule are specified (Section 4.15), but the two reviewers have not been nominated and no review has taken place.

The lawful policy-transition semantics are author-approved for protocol preparation, but the repaired scenario is a contract test, not a rate estimate, and the frozen scenario matrix still requires human approval. High deterministic recall on a synthetic generator (Section 6.2) does not imply that rules would find nearly every real error.

7.6 Temporal Validity

Policy-aware analysis is only as correct as the supplied policy history; applying today's rule to yesterday's decision can create false violations (Sections 2.6 and 3.7). Bitemporal storage supports reconstruction but does not decide the organization's substantive policy, which requires human approval and migration planning.

7.7 Explanation and Human Factors

Leaving authorization with HR reduces the risk of an anomaly score becoming an automatic employment decision, but it does not remove automation bias: reviewers may over-trust a numerical score or a polished explanation. The optional usability study could compare binary eligibility indicators with evidence-rich explanations using interpretation accuracy, completion time, and usability ratings. It was not conducted; no participants were recruited and no approved protocol was run.

7.8 Privacy, Security, and Fairness

The experiments use fictional records and need no cloud processing. Local-first deployment can support data sovereignty, but offline storage is not secure by default; access control, backups, audit integrity, and device protection still matter.

The study makes no fairness claim across protected demographic groups. Rare-legitimate false alerts are a useful diagnostic, but demographic fairness requires lawful attributes, a defined harm model, adequate representation, and a separate analysis. The

system must not infer sensitive traits or use anomaly ranking as a proxy for disciplinary action.

7.9 Reproducibility Versus Validity

Checksums, manifests, source hashes, CI, and independent arithmetic audits make the work reproducible, but they cannot make an invalid estimand valid. Their main value in this study was to show exactly what was calculated and why the central comparison failed.

7.10 Approved Redesign and Remaining Freeze Gate

The approved primary design keeps a residual task after removing only invariant contradictions and adds policy context that the gate does not consume: historical policy identity, effective versus recorded timing, recognized-service evidence, jurisdiction, and correction provenance. The end-to-end rules-plus-ML queue remains secondary; it measures the system a client would use, but rules can dominate it and hide the incremental contribution of ML.

The invariant rule boundary, research bitemporal schema, linked entitlement, temporal views, compatibility guard, equal-capacity endpoint, and zero-positive treatment are implemented and tested in development code. The 900-scenario diagnostic exercised the repeated-seed pipeline, but its wrong-policy confound (Section 6.8) means it does not satisfy the freeze gate. The exact balanced scenario matrix, protocol-scale correction examples, three-test Holm implementation, dependencies, source revision, review execution, and protocol still need independent verification and freeze before protected generation.

Because the approval refined the hypothesis after development diagnostics were inspected, those diagnostics remain exploratory. New development and validation runs must test the redesigned pipeline before the specification, features, endpoint, capacity, seeds, and analysis are frozen.

7.11 Practical Implications for MyHR

Historical-policy handling remains a product requirement even though the `8/72` false-alert reading was invalidated (Section 6.6). A client importing old promotion records needs the system to distinguish the policy effective at the decision date from the policy active today. Without that evidence, MyHR should show an indeterminate status and request review instead of asserting a violation, and import templates would need policy identifiers, effective and recorded dates, and evidence for recognized service or corrections. Explanations should likewise show which facts are present, which assumptions were inferred, and which evidence is missing, so that an incomplete history never becomes a confident personnel judgement. This also yields better research data, because future labels can distinguish missing evidence from confirmed error.

8 Conclusion and Future Work

This study asked whether explicit workforce-progression policy context improves anomaly detection beyond raw administrative features and deterministic validation. MyHR now contains the technical basis for a defensible investigation (Section 1.5).

The development experiments do not establish the expected advantage. They first showed how a reasonable-looking comparison can fail, through generator-detector

coupling and the collapse of policy features in the historical residual design. In the repaired artifact `4feea4508de1bb05`, raw and policy-aware residual queues each reviewed 96 records (Section 6.7); raw found 36/108 anomalies and policy-aware 39/108, with a recall difference of +0.0278, 95% CI [-0.0278, +0.1019], sign-flip p=0.6875. Wrong-policy context selected the same records as policy-matched context, and representations that add a single policy feature to the common core found 42 anomalies, so no benefit can be attributed to correct policy knowledge. In the 900-scenario diagnostic `8f1494e4abf2d268`, wrong-policy features reproduced about 82% of the apparent recall gain, and its full audit is incomplete. Deterministic policy rules flagged 0 of 72 generated rare-legitimate labels in the repaired artifact, but 24 of these labels lack stored employee-specific entitlement, and the earlier `39/72` and `8/72` readings are not lawful-case false-positive rates (Section 6.6).

Several steps remain before a confirmatory study: a raw control with added non-policy columns of comparable variance, the blinded realism review, independent validation, a full artifact audit, and a dated freeze of the scenario matrix, dependencies, analysis, and source revision. Only after separate human approval may the locked holdout be run, once. Duplicate detection remains a secondary experiment, and evidence-rich explanation remains a product contribution; neither replaces a valid central experiment. The paper offers a reproducible platform, a documented diagnosis of a failed comparison, and a path to a confirmatory study.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in 2008 Eighth IEEE International Conference on Data Mining, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation-Based Anomaly Detection,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 1, pp. 1–39, 2012. DOI: 10.1145/2133360.2133363.

[3] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, “ADBench: Anomaly Detection Benchmark,” in Advances in Neural Information Processing Systems 35, 2022, pp. 32142–32159. DOI: 10.52202/068431-2329.

[4] N. Vaska, K. Leahy, and V. Helus, “Context-Dependent Anomaly Detection with Knowledge Graph Embedding Models,” in 2022 IEEE 18th International Conference on Automation Science and Engineering (CASE), 2022, pp. 2020–2027. DOI: 10.1109/CASE49997.2022.9926631.

[5] Z. Li, Y. Zhu, and M. van Leeuwen, “A Survey on Explainable Anomaly Detection,” ACM Transactions on Knowledge Discovery from Data, vol. 18, no. 1, pp. 1–54, 2024. DOI: 10.1145/3609333.

[6] M. Carletti, M. Terzi, and G. A. Susto, “Interpretable Anomaly Detection with DIFFI: Depth-Based Feature Importance of Isolation Forest,” Engineering Applications of Artificial Intelligence, vol. 119, art. 105730, 2023. DOI: 10.1016/j.engappai.2022.105730.

[7] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” PLOS ONE, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432.

[8] S. Sørbø and M. Ruocco, “Navigating the Metric Maze: A Taxonomy of Evaluation Metrics for Anomaly Detection in Time Series,” Data Mining and Knowledge Discovery, vol. 38, no. 3, pp. 1027–1068, 2024. DOI: 10.1007/s10618-023-00988-8.

[9] D. Fourure, M. U. Javaid, N. Posocco, and S. Tihon, “Anomaly Detection: How to Artificially Increase Your F1-Score with a Biased Evaluation Protocol,” in Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track, Lecture Notes in Computer Science, Springer, 2021, pp. 3–18. DOI: 10.1007/978-3-030-86514-6_1.

[10] P. Röchner, S. Klüttermann, K. Kammler, F. Rothlauf, E. Müller, and D. Schlör, “We Need to Rethink Benchmarking in Anomaly Detection,” arXiv preprint arXiv:2507.15584, 2025. DOI: 10.48550/arXiv.2507.15584.

[11] G. Steinbuss and K. Böhm, “Benchmarking Unsupervised Outlier Detection with Realistic Synthetic Data,” ACM Transactions on Knowledge Discovery from Data, vol. 15, no. 4, pp. 1–20, 2021. DOI: 10.1145/3441453.

[12] L. Hansen, N. Seedat, M. van der Schaar, and A. Petrovic, “Reimagining Synthetic Tabular Data Generation through Data-Centric AI: A Comprehensive Benchmark,” in Advances in Neural Information Processing Systems 36, 2023, pp. 33781–33823. DOI: 10.52202/075280-1466.

[13] G. Neelamegam, J. Bhogal, P. Samimi, and Ö. Öztürkoğlu, “Comprehensive Insights into Bitemporal Databases: A PRISMA-Guided Systematic Literature Review,” Journal of Data, Information and Management, vol. 8, no. 2, pp. 71–96, 2026. DOI: 10.1007/s42488-026-00162-x.

[14] Z. Deng, X. Xuan, K.-L. Ma, and Z. Kong, “A Reliable Framework for Human-in-the-Loop Anomaly Detection in Time Series,” ACM Transactions on Interactive Intelligent Systems, vol. 16, no. 2, pp. 1–23, 2026. DOI: 10.1145/3778167.

[15] T. Wang, H. Lin, X. Han, X. Chen, B. Cao, and L. Sun, “Towards Universal Dense Blocking for Entity Resolution,” arXiv preprint arXiv:2404.14831, 2024. DOI: 10.48550/arXiv.2404.14831.

[16] J. Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets,” Journal of Machine Learning Research, vol. 7, pp. 1–30, 2006. Available: https://www.jmlr.org/papers/v7/demsar06a.html (accessed 1 November 2026).

[17] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979. Available: https://www.jstor.org/stable/4615733 (accessed 1 November 2026).

[18] K. Krippendorff, “Reliability in Content Analysis: Some Common Misconceptions and Recommendations,” Human Communication Research, vol. 30, no. 3, pp. 411–433, 2004. DOI: 10.1111/j.1468-2958.2004.tb00738.x.

Appendix: Reproduction

Code availability. The source code, tests, figure scripts, and documentation are in the repository https://github.com/ibrahimify/MyHR (branch `research`, tag `tdk-2026-submission`). The research artifacts named in this paper are excluded from Git because of their size (up to 1.9 GB for the 900-scenario run). Their identifiers, source revisions, and SHA-256 manifests are given in the text, and the author can provide the artifacts on request.

Settings. Isolation Forest is fitted with `contamination="auto"` and a recorded model seed per scenario. The primary review budget is 10% of the residual test histories, rounded up, for every representation. Dataset, split, and model seeds, the scenario configurations, and the package versions (Python 3.13.5, scikit-learn 1.9.1, NumPy 2.5.4) are stored in each artifact (`metadata.json`, `scenario_configs.csv`). Scenario seeds are the resampling unit for all intervals and tests. The in-app demonstration uses 25 trees.

Commands, run from the repository root in PowerShell:

```
pip install -r requirements.txt -r requirements-research.txt
python -m unittest discover -s tests
python scripts/verify_tdk_citations.py --require-local-artifact
$r = "research_outputs/independent_development"
$d = "independent_development_4feea4508de1bb05"
python -m scripts.audit_tdk_protocol_artifact "$r/$d"
python scripts/build_tdk_figures.py
python scripts/verify_anomaly_demo.py
```